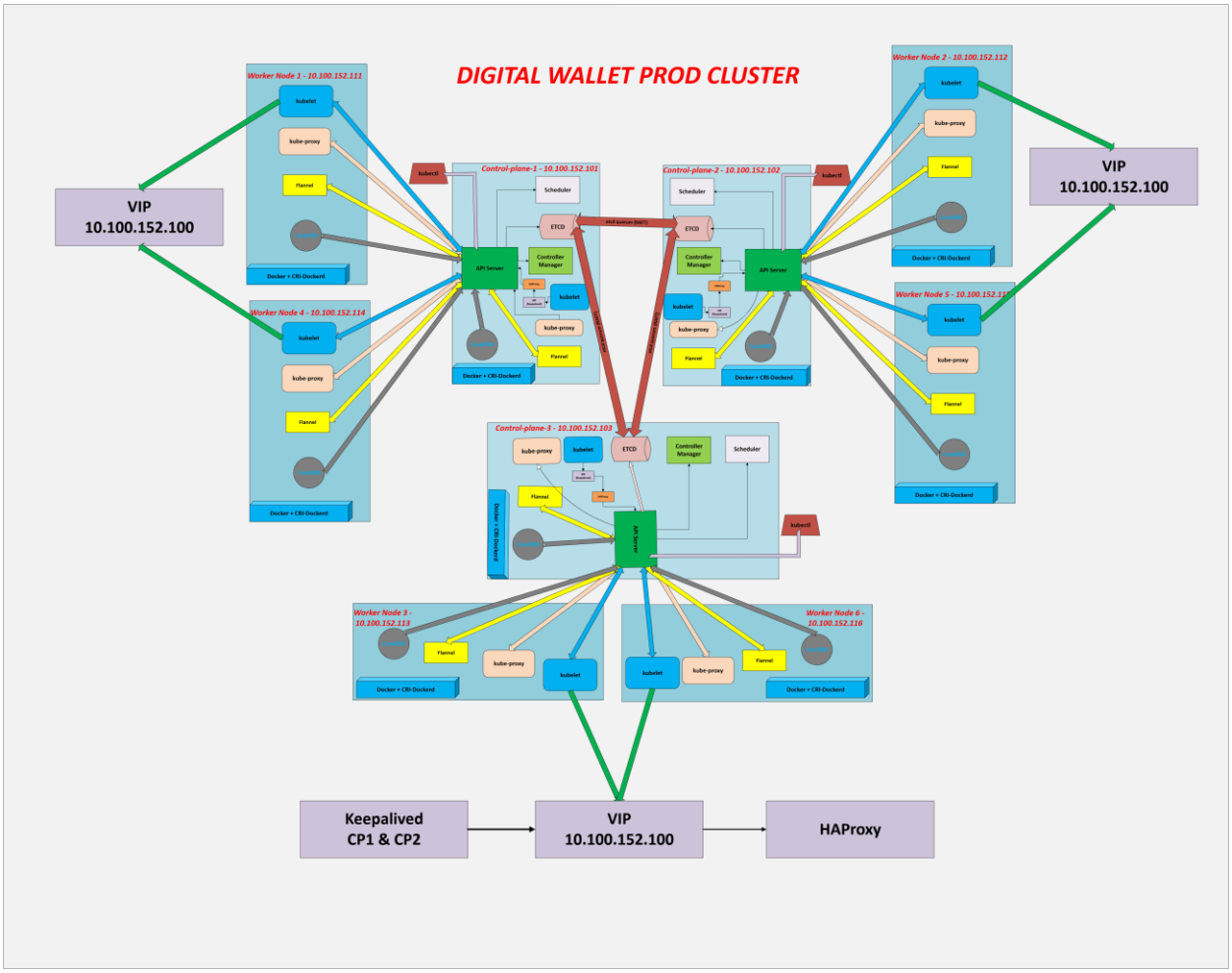
Digital Wallet

Kubernetes Setup | Production | PLAN

Documentation is a **production-grade, step-by-step** plan to bootstrap a **fresh Kubernetes v1.33.6** cluster with **3 control planes** and **6 workers(Depends on load)**, using **Docker Engine + cri-dockerd** as the runtime, and an **HA API** via **HAProxy + Keepalived -** designed for "zero-downtime" operations during maintenance and app rollouts.

Digital Wallet Proposing Production K8S cluster architecture:



VISIO Link: [DW-PROD-K8S-Cluster-Arch]

NOTE:

- ❖ Kubernetes v1.33.6 and kubeadm config API v1beta4 are used in this document based on validated TEST and UAT environments.
- ❖ These versions are provided via our internal / vendor-supported Kubernetes distribution and may not yet be reflected in upstream Kubernetes documentation.
- ❖ All procedures in this guide have been tested and confirmed working in non-production environments prior to production rollout.

Assumptions:

- OS: RHEL9
- Packages availability: docker, curl, yq, dnf
- **IPs: IPs using only for Documentation purpose**
    - **Control planes:**
        - `CP-1: 10.100.152.101`
        - `CP-2: 10.100.152.102`
        - `CP-3: 10.100.152.103`
    - **Workers Nodes:**
        - `WN-1: 10.100.152.111`
        - `WN-2: 10.100.152.112`
        - `WN-3: 10.100.152.113`
        - `WN-4: 10.100.152.114`
        - `WN-5: 10.100.152.115`
        - `WN-6: 10.100.152.116`
- All servers have **internet** and **internal SSH**.
- We'll deploy **stacked etcd** on the 3 control planes (the recommended, simpler HA topology for kubeadm).

Why these choices?

- **kubeadm + stacked etcd** is the standard HA approach for 3 (or more) control planes.
- **Docker Engine** requires **cri-dockerd** because Kubernetes removed the built-in dockershim; cri-dockerd is the supported adapter.
- We'll front the API with **HAProxy + Keepalived (VIP)**, as documented in kubeadm HA guidance.

Versions & prerequisites

- **Kubernetes:** v**1.33.6** (stable patch; v1.33 is the April 2025 release line).

- **kubeadm config API:** use the **latest** kubeadm config API for your minor version (newer kubeadm deprecates older `v1beta3`; we'll use the current API in 1.33). If your tooling complains, run `kubeadm config migrate` to the supported API.
- **Ports open** between nodes:
    - `6443` (API), `2379-2380` (etcd), `10250` (kubelet), `10257/10259` (controller/scheduler), plus CNI ports as needed. (Common guidance).
- **Disable swap, enable IP forwarding, br_netfilter** on **all nodes** (control & worker).
- **CNI**: choose Calico/Flannel/Cilium; cri-dockerd defaults to **CNI** on Linux, so you need a CNI plugin. We can use – Flannel as used in TEST and UAT environments

1) HA API: Keepalived + HAProxy (2 small LB nodes or colocate on CPs)

We can either:

- **Colocate** HAProxy+Keepalived **on CP-1 and CP-2**, or
- Use **two dedicated LB nodes**.

Below assumes **colocation on CP-1 and CP-2** and a **VIP = 10.100.152.100**

1.1 Install packages (on CP-1 and CP-2)

```
sudo dnf install -y keepalived haproxy psmisc
```

(Using HAProxy + Keepalived for a TCP VIP is a common approach for kubeadm HA.)

1.2 Keepalived config (on both CP-1 & CP-2)

`/etc/keepalived/keepalived.conf` (adjust NIC `eth0` if different)

```
vrrp_instance VI_1 {
  state MASTER          # BACKUP on CP-2
  interface eth0
  virtual_router_id 51
  priority 110          # use 100 on CP-2
  advert_int 1
  authentication {
    auth_type PASS
    auth_pass kubeHA
  }
```

```
  virtual_ipaddress {
    10.100.152.100/24
  }
}
```

```
sudo systemctl enable --now keepalived
ip a | grep 10.100.152.100   # VIP should appear on CP-1
```

(Guided by HA considerations that recommend a VIP plus redundant LBs.)

1.3 HAProxy config (on both CP-1 & CP-2)

`/etc/haproxy/haproxy.cfg`

```
global
  log /dev/log local0
  log /dev/log local1 notice
  daemon


defaults
  log global
  mode tcp
  option tcplog
  timeout connect 10s
  timeout client  1m
  timeout server  1m


frontend kubernetes-api
  bind *:6443
  default_backend kubernetes-masters


backend kubernetes-masters
  balance roundrobin
  option tcp-check
  server cp1 10.100.152.101:6443 check
  server cp2 10.100.152.102:6443 check
  server cp3 10.100.152.103:6443 check
```

```
sudo systemctl enable --now haproxy
ss -lntp | grep 6443   # should show haproxy listening on VIP:6443
```

(Typical HAProxy+Keepalived setup for kube-apiserver LB.)

2) Install Docker Engine + cri-dockerd on all nodes

Kubernetes no longer includes dockershim; **cri-dockerd** provides the CRI adapter to use Docker Engine

2.1 Docker Engine

Use official Docker CE install for your OS (summarized):

```
sudo dnf -y install yum-utils
sudo yum-config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io
sudo systemctl enable --now docker
```

2.2 cri-dockerd

Install via released packages/binaries and enable service:

```
# Download latest release (example; pick the right arch/version)
wget https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.14/cri-dockerd-0.3.14-1.el9.x86_64.rpm
sudo dnf install -y ./cri-dockerd-0.3.14-1.el9.x86_64.rpm

# Enable service
sudo systemctl enable --now cri-docker.service
sudo systemctl status cri-docker.service
```

(Use the official install docs / releases page for your platform.)

CRI socket for kubelet/kubeadm with Docker via cri-dockerd is:
`unix:///var/run/cri-dockerd.sock` (we'll set it in kubeadm).

3) Prepare all nodes (sysctl, kernel modules)

```
sudo swapoff -a
sudo sed -ri '/\sswap\s/s/^/#/' /etc/fstab

# Kernel modules & sysctls
sudo modprobe br_netfilter
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
EOF
sudo sysctl --system
```

4) Install Kubernetes tooling v1.33.6 on all nodes

Install **kubeadm, kubelet, kubectl** matching **v1.33.6**, then **hold**:

```
# RHEL: use Kubernetes repo
cat <<EOF | sudo tee /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.kubernetes.io/core:/stable:/v1.33/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.kubernetes.io/core:/stable:/v1.33/rpm/repodata/repomd.xml.key
EOF

sudo dnf install -y kubeadm-1.33.6 kubelet-1.33.6 kubectl-1.33.6
sudo systemctl enable --now kubelet
```

5) Initialize the first control plane (CP-1)

Create `/root/kubeadm-config.yaml` on **CP-1**:

Note: kubeadm config API evolves; for 1.33 you should use the **current** API (`v1beta4` or newer as provided by kubeadm in this line). If you have an older file, run `kubeadm config migrate`

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
kubernetesVersion: v1.33.6
clusterName: prod-cluster
controlPlaneEndpoint: "10.100.152.100:6443"   # VIP via HAProxy
networking:
  podSubnet: "10.244.0.0/16"              # if using Flannel; for Calico, commonly
192.168.0.0/16
  serviceSubnet: "10.96.0.0/12"
apiServer:
  certSANs:
    - "10.100.152.100"
    - "10.100.152.101"
    - "10.100.152.102"
    - "10.100.152.103"
controllerManager: {}
scheduler: {}
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
nodeRegistration:
  criSocket: "unix:///var/run/cri-dockerd.sock"
```

Initialize: sudo kubeadm init --config /root/kubeadm-config.yaml

- Save the control-plane and worker `kubeadm join` commands shown.
- Set kubectl for admin:

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Apply Network Add-on

`kubectl apply -f https://github.com/flannel-io/flannel/releases/download/v0.25.5/kube-flannel.yml`

Confirm all control-plane components and CNI pods become **Ready**.

7) Upload shared certs and **join** CP-2 and CP-3
On **CP-1**:
kubeadm init phase upload-certs --upload-certs
Capture the "certificate-key" printed
kubeadm token create --print-join-command

On **CP-2** and **CP-3** (replace placeholders):

```
sudo kubeadm join 10.100.152.100:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <certificate-key> \
  --cri-socket unix:///var/run/cri-dockerd.sock
```

kubectl get nodes -o wide

You should see **three control planes**.

8) Join the **six workers**
On each **WN-1 … WN-6**:
sudo kubeadm join 10.100.152.100:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --cri-socket unix:///var/run/cri-dockerd.sock
``
kubectl get nodes

Node type	Required before kubeadm
Control Plane	docker + cri-dockerd + kubelet + kubeadm + kubectl
Worker Node	docker + cri-dockerd + kubelet + kubeadm (**NO kubectl required**)

`kubelet` must already be running on **every node**, otherwise `kubeadm join` **will fail**

This is per kubeadm design.

COMMON BASE STEPS (ALL NODES)

Run on ALL servers (CP + Workers)

1. Disable swap

swapoff -a

sed -i '/ swap / s/^/#/' /etc/fstab

2. Kernel modules & sysctl

modprobe br_netfilter

cat <<EOF >/etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
EOF

sysctl --system

Required for **kube-proxy, CNI networking**

Mandatory per Kubernetes docs

Docker + cri-dockerd

systemctl status docker

systemctl status cri-docker

ls -l /var/run/cri-dockerd.sock

If socket exists, runtime is ready.

CONTROL PLANE NODE SETUP (CP-1, CP-2, CP-3)

Run these **ONLY on control-plane nodes**

4. Install Kubernetes control-plane packages
dnf install -y kubelet-1.33.6 kubeadm-1.33.6 kubectl-1.33.6

Enable kubelet:
systemctl enable --now kubelet
systemctl restart --now kubelet

kubelet starts but stays in **crash-loop** until kubeadm runs — this is EXPECTED.

5. Why kubectl is needed on Control Planes

- Used for:
  - Cluster admin
  - CNI installation
  - Validation
  - Troubleshooting
- Not required on workers

Now control-plane is READY for:

kubeadm init
or
kubeadm join --control-plane

WORKER NODE SETUP (WN-1 … WN-6)

Run these **ONLY on worker nodes**

6. Install Kubernetes worker components
dnf install -y kubelet-1.33.6 kubeadm-1.33.6

Do NOT install kubectl on workers

Enable kubelet:

systemctl enable --now kubelet
systemctl restart --now kubelet

Again — kubelet will run but not register until join

7. Verify BEFORE joining

systemctl status kubelet

ls -l /var/run/cri-dockerd.sock
If both pass → node is join-ready

8. Join Worker Node to Cluster

Now (ONLY now) run:

```
kubeadm join 10.100.152.100:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --cri-socket unix:///var/run/cri-dockerd.sock
```

📌 Uses:

- HA VIP
- cri-dockerd socket
- kubelet already running

VERIFICATION (From Control Plane)

kubectl get nodes -o wide

VERY COMMON ERRORS (Why Join Fails)

Error	Root cause
`kubeadm join hangs`	kubelet not installed / not running
`CRI socket error`	cri-dockerd not installed / wrong socket
`node not Ready`	CNI not installed yet
`container runtime not detected`	Docker running but cri-dockerd missing

Automation setup:

production-ready shell scripts for **RHEL 9.6** to build your **Kubernetes v1.33.6** cluster using **Docker Engine + cri-dockerd** and **Flannel** (CNI).
They're split by role and phase so you can run them in order or automate them via Ansible/Jenkins.

Why these steps?
Kubernetes removed the built-in dockershim; **cri-dockerd** is the supported adapter to use Docker Engine with Kubernetes (Kubelet talks CRI to cri-dockerd). You must also set base kernel/sysctl and install kubelet/kubeadm before `init/join`. Flannel provides Pod networking (CNI)

How to use

1. Adjust variables in each script (**versions, VIP, pod CIDR, NIC**).
2. **Run 01** on **all** nodes.
3. **Run 02** on **control planes**; **Run 03** on **workers**.
4. On **CP-1**: **Run 04** (init), then **Run 06** (Flannel).
5. On **CP-2 / CP-3**: **Run 05** (join control plane).
6. On **workers**: **Run 07** (join worker).
7. (Optional) **Run 08/09** for **Keepalived + HAProxy** VIP if you want HA API with a floating IP on CP-1 & CP-2.

These scripts assume **kubeadm HA with stacked etcd** on 3 control planes (recommended and simpler).

Variables (copy/paste into each script or source from `vars.env`)

```
# vars.env (edit to match your environment)
K8S_VERSION="1.33.6"
POD_CIDR="10.244.0.0/16"            # Flannel default
VIP_API="10.100.152.100:6443"       # HA VIP (if using HAProxy+Keepalived)
CP1_IP="10.100.152.101"
CP2_IP="10.100.152.102"
CP3_IP="10.100.152.103"
NIC="eth0"                          # NIC for Keepalived
CRI_SOCKET="unix:///var/run/cri-dockerd.sock"
# Choose cri-dockerd package matching RHEL 9.6 (adjust if newer released):
```

CRI_DOCKERD_RPM_URL="https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.14/cri-dockerd-0.3.14-1.el9.x86_64.rpm"
``

01) Common base setup (run on all nodes)

Disables swap, sets netfilter/ip_forward, installs Docker CE and cri-dockerd, configures Kubernetes repo

```
#!/usr/bin/env bash
set -euo pipefail


source ./vars.env


echo "[Common] Disabling swap..."
swapoff -a
sed -ri '/\sswap\s/s/^/#/' /etc/fstab


echo "[Common] Kernel modules & sysctl..."
modprobe br_netfilter || true
cat <<EOF >/etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
EOF
sysctl --system


echo "[Common] Install dependencies..."
dnf -y install yum-utils curl wget gnupg2


echo "[Common] Install Docker CE..."
yum-config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
dnf install -y docker-ce docker-ce-cli containerd.io
systemctl enable --now docker
systemctl status docker --no-pager
```

```
echo "[Common] Install cri-dockerd..."
wget -O /tmp/cri-dockerd.rpm "$CRI_DOCKERD_RPM_URL"
dnf install -y /tmp/cri-dockerd.rpm
systemctl enable --now cri-docker
systemctl status cri-docker --no-pager
ls -l /var/run/cri-dockerd.sock


echo "[Common] Kubernetes repo (v$K8S_VERSION stream)..."
cat <<'EOF' >/etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.kubernetes.io/core:/stable:/v1.33/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.kubernetes.io/core:/stable:/v1.33/rpm/repodata/repomd.xml.key
EOF


echo "[Common] Hold packages to specific version: v$K8S_VERSION will be set in role scripts."
echo "[Common] Base setup completed."
```

Notes

- Using cri-dockerd is required to use Docker with Kubernetes 1.24+ and later.
- You must have a **CNI plugin**; Flannel will be installed later

02) Control plane node components (run on CP-1, CP-2, CP-3)

Installs `kubeadm`, `kubelet`, **`kubectl`** (kubectl is useful on control planes), enables kubelet

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env


echo "[CP] Install kubeadm/kubelet/kubectl v$K8S_VERSION..."
dnf install -y kubeadm-${K8S_VERSION} kubelet-${K8S_VERSION} kubectl-
${K8S_VERSION}
```

```
echo "[CP] Enable kubelet..."
systemctl enable --now kubelet
systemctl status kubelet --no-pager || true

echo "[CP] Control-plane prerequisites done. kubelet will crash-loop until kubeadm init/join."
```

03) Worker node components (run on WN-1 … WN-6)

Installs `kubeadm`, `kubelet` (no `kubectl` needed), enables kubelet. (Workers must have kubelet running before join).

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env

echo "[Worker] Install kubeadm/kubelet v$K8S_VERSION..."
dnf install -y kubeadm-${K8S_VERSION} kubelet-${K8S_VERSION}

echo "[Worker] Enable kubelet..."
systemctl enable --now kubelet
systemctl status kubelet --no-pager || true

echo "[Worker] Worker prerequisites done."
```

04) Initialize the first control plane (run only on CP-1)

Creates a kubeadm config using the HA **VIP** (`controlPlaneEndpoint`), sets cri-dockerd socket, and Flannel pod CIDR; then runs `kubeadm init`

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env


echo "[CP-1] Writing kubeadm config..."
cat <<EOF >/root/kubeadm-config.yaml
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
kubernetesVersion: v${K8S_VERSION}
clusterName: prod-cluster
controlPlaneEndpoint: "${VIP_API}"   # HA VIP via HAProxy
networking:
  podSubnet: "${POD_CIDR}"
  serviceSubnet: "10.96.0.0/12"
apiServer:
  certSANs:
    - "$(echo ${VIP_API} | cut -d: -f1)"
    - "${CP1_IP}"
    - "${CP2_IP}"
    - "${CP3_IP}"
controllerManager: {}
scheduler: {}
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
nodeRegistration:
  criSocket: "${CRI_SOCKET}"
EOF


echo "[CP-1] kubeadm init..."
kubeadm init --config /root/kubeadm-config.yaml


echo "[CP-1] Configure kubectl..."
mkdir -p $HOME/.kube
cp -f /etc/kubernetes/admin.conf $HOME/.kube/config
chown $(id -u):$(id -g) $HOME/.kube/config
```

```
echo "[CP-1] Upload certs for multi-control-plane join..."
kubeadm init phase upload-certs --upload-certs

echo "[CP-1] Print join commands..."
kubeadm token create --print-join-command

echo "[CP-1] Init complete. Next: install Flannel (script 06) and join CP-2/CP-3 (script 05)."
```

If kubeadm warns an older config API, use `kubeadm config migrate` to the latest supported API for 1.33; kubeadm prints deprecation hints

05) Join additional control planes (run on CP-2 and CP-3)

Requires the `--token`, `--discovery-token-ca-cert-hash`, and `--certificate-key` printed by CP-1. (Use the HA VIP and cri socket.) [kubernetes.io]

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env

JOIN_TOKEN="${JOIN_TOKEN:-<set-from-cp1-output>}"
DISC_HASH="${DISC_HASH:-sha256:<set-from-cp1-output>}"
CERT_KEY="${CERT_KEY:-<set-from-cp1-output>}"

echo "[CP-n] Joining control plane using VIP ${VIP_API}..."
kubeadm join ${VIP_API} \
  --token ${JOIN_TOKEN} \
  --discovery-token-ca-cert-hash ${DISC_HASH} \
  --control-plane \
  --certificate-key ${CERT_KEY} \
  --cri-socket ${CRI_SOCKET}
```

```
echo "[CP-n] Control plane joined."
```

06) Install Flannel CNI (run on CP-1 after init)

Flannel is a CNI; apply the manifest with the matching **pod CIDR**. (You can host internally or fetch from upstream. Example below uses upstream typical manifest.)

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env

echo "[CNI] Installing Flannel..."
# Official Flannel install usually:
# https://github.com/flannel-io/flannel/releases/download/v0.25.6/kube-flannel.yml
# Ensure it matches your POD_CIDR=10.244.0.0/16 (default in the manifest).
curl -fsSL -o /tmp/kube-flannel.yml \
  https://github.com/flannel-io/flannel/releases/download/v0.25.6/kube-flannel.yml

kubectl apply -f /tmp/kube-flannel.yml

echo "[CNI] Waiting for flannel pods..."
kubectl -n kube-flannel get pods
echo "[CNI] Flannel applied."
```

cri-dockerd defaults to **CNI** plugin usage; without a CNI, pods won't get network

07) Join worker nodes (run on each worker)

After CNI is installed and control planes healthy, join workers using the VIP and cri socket.

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env
```

```
JOIN_TOKEN="${JOIN_TOKEN:-<set-from-cp1-output>}"
DISC_HASH="${DISC_HASH:-sha256:<set-from-cp1-output>}"

echo "[Worker] Join cluster using VIP ${VIP_API}..."
kubeadm join ${VIP_API} \
  --token ${JOIN_TOKEN} \
  --discovery-token-ca-cert-hash ${DISC_HASH} \
  --cri-socket ${CRI_SOCKET}

echo "[Worker] Joined cluster."
```

(Optional) 08) Keepalived (VIP) config (run on CP-1 and CP-2)

Provides a floating VIP for API, recommended by kubeadm HA considerations; pair it with HAProxy.

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env

echo "[LB] Installing keepalived..."
dnf install -y keepalived

STATE="MASTER"    # On CP-1; set BACKUP on CP-2
PRIORITY="110"    # Use 100 on CP-2

cat <<EOF >/etc/keepalived/keepalived.conf
vrrp_instance VI_1 {
  state ${STATE}
  interface ${NIC}
  virtual_router_id 51
  priority ${PRIORITY}
  advert_int 1
  authentication {
    auth_type PASS
    auth_pass kubeHA
  }
```

```
  virtual_ipaddress {
    $(echo ${VIP_API} | cut -d: -f1)/24 dev ${NIC}
  }
}
EOF


systemctl enable --now keepalived
ip a | grep "$(echo ${VIP_API} | cut -d: -f1)" || true
echo "[LB] Keepalived configured."
```

(Optional) 09) HAProxy (API LB) config (run on CP-1 and CP-2)

HAProxy binds to the VIP and forwards to each control plane port 6443

```
#!/usr/bin/env bash
set -euo pipefail
source ./vars.env


echo "[LB] Installing haproxy..."
dnf install -y haproxy psmisc


cat <<EOF >/etc/haproxy/haproxy.cfg
global
  log /dev/log local0
  log /dev/log local1 notice
  daemon


defaults
  log global
  mode tcp
  option tcplog
  timeout connect 10s
  timeout client  1m
  timeout server  1m
```

```
frontend kubernetes-api
  bind $(echo ${VIP_API} | cut -d: -f1):$(echo ${VIP_API} | cut -d: -f2)
  default_backend kubernetes-masters

backend kubernetes-masters
  balance roundrobin
  option tcp-check
  server cp1 ${CP1_IP}:6443 check
  server cp2 ${CP2_IP}:6443 check
  server cp3 ${CP3_IP}:6443 check
EOF

systemctl enable --now haproxy
ss -lntp | grep $(echo ${VIP_API} | cut -d: -f2) || true
echo "[LB] HAProxy configured."
```

Post-build checks

Run on CP-1:

```
kubectl get nodes -o wide
kubectl -n kube-flannel get pods
```

HA tests:

- Stop kube-apiserver on one CP → `kubectl get nodes` still works via VIP.
- Stop Keepalived on CP-1 → VIP should move to CP-2.
- With 3 CPs (stacked etcd), stopping one CP maintains quorum.

“Zero-downtime” app rollouts (Kubernetes settings)

When you deploy your microservices, add:

- **Readiness & liveness probes**
- **RollingUpdate** tuned for 0 downtime:

```
strategy:
```

```
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

PodDisruptionBudget to protect replicas during drains/upgrades

UAT vs Production k8s setup

What exactly changes in setup, commands, order, and configuration between

- **1 Control Plane + 1 Worker**
- **3 Control Planes + multiple Workers**

Below is a **step-by-step, command-level comparison**, focusing on **installation flow, kubeadm behavior, join logic, and core config differences**.

No theory fluff — **only practical differences**.

Mental Model First (Very Important)

Aspect	1 CP + 1 WN	3 CP + N WN
kubeadm init	✅ Once	✅ Once (CP-1 only)
kubeadm join	Worker only	Control-plane **and** workers
controlPlaneEndpoint	❌ Not needed	✅ Mandatory
cert upload	❌ Not needed	✅ Mandatory
HA LB / VIP	❌ Not needed	✅ Mandatory
etcd	Single	Quorum (stacked)

Keep this table in mind — everything below maps to it.

1. Installation Steps — What Changes?

Common steps (IDENTICAL in both setups)

These steps **do NOT change**:

Shell
swapoff -a

modprobe br_netfilter

sysctl --system

install docker + cri-dockerd

install kubelet + kubeadm

systemctl enable --now kubelet

Same commands
Same order
Same on all nodes

Difference starts at kubeadm init

2. kubeadm init — Core Difference

1 CP + 1 WN

Command

Shell
kubeadm init --pod-network-cidr=10.244.0.0/16 --cri-socket unix:///var/run/cri-dockerd.sock

Characteristics

- No HA
- No VIP
- No cert sharing
- etcd is **single node**
- API endpoint = CP IP

kubeadm auto-generates everything locally.

3 CP + N WN (BIG DIFFERENCE)

We MUST use a config file

```
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
controlPlaneEndpoint: "10.100.152.100:6443"
kubernetesVersion: v1.33.6
networking:
        podSubnet: 10.244.0.0/16
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
nodeRegistration:
        criSocket: unix:///var/run/cri-dockerd.sock
```

Command

Command: kubeadm init --config kubeadm-config.yaml

What changed (core):

Item	1 CP	3 CP
controlPlaneEndpoint	❌	✅
VIP/LB	❌	✅
Cert SANs	Optional	✅ Required
Future CP join	❌	✅ Designed

✅ This is the **single most important difference**.

3. Certificate Handling — MAJOR CHANGE

1 CP

- Certificates stay local
- No sharing needed
- No extra steps

3 CP (Mandatory step)

After init **ONLY on CP-1**:

Shell
kubeadm init phase upload-certs --upload-certs

You now get:

--certificate-key <KEY>

📌 This key is **required** to join other control planes.

4. kubeadm join — Completely Different Behavior

Worker Join (Same in both)

kubeadm join <endpoint>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash> --cri-socket unix:///var/run/cri-dockerd.sock

✅ No difference here.

Control Plane Join (ONLY in 3 CP setup)

This does NOT exist in 1 CP setups.

kubeadm join 10.100.152.100:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash> --control-plane --certificate-key <cert-key> --cri-socket unix:///var/run/cri-dockerd.sock

Core differences:

Flag	Purpose
`--control-plane`	Tells kubeadm to install CP components
`--certificate-key`	Fetch shared certs securely
API endpoint	Must be VIP
etcd	Joins quorum

❌ Missing any of these = join failure.

5. etcd Behavior — Hidden but Critical Change

1 CP

- etcd static pod on CP
- No peer communication
- No quorum

Shell

```
kubectl get pods -n kube-system | grep etcd
```

etcd-cp-1

3 CP

- etcd runs on **every CP**

- Peer-to-peer communication
- Quorum enforced

Shell

etcd-cp-1

etcd-cp-2

etcd-cp-3

📌 kubeadm auto-handles this **only if controlPlaneEndpoint exists**.

6. Load Balancer / VIP — EXISTS ONLY IN 3 CP

Area	1 CP	3 CP
HAProxy	❌	✅
Keepalived	❌	✅
VIP	❌	✅
API endpoint	CP IP	VIP

This directly affects:

- kubeadm init
- kubeadm join
- kubeconfig
- CI/CD access

7. Node Commands — Side-by-Side Summary

1 CP + 1 WN

CP:
- kubeadm init
- install CNI

WN:
- kubeadm join (worker)

3 CP + N WN

CP-1:
- kubeadm init (with config)
- upload certs
CP-2, CP-3:
- kubeadm join --control-plane

Workers:
- kubeadm join (worker)

8. Failure Behavior (Why Commands Matter)

Action	1 CP	3 CP
Stop CP	Cluster dead	Cluster alive
Restart API	kubectl fails	Transparent
Upgrade CP	Full outage	Rolling

📌 These behaviors are **direct results of setup differences above**, not magic.

Final One-Line Summary

1 CP setup is "single-node kubeadm init"
3 CP setup is "cluster-aware kubeadm orchestration with cert sharing, LB, and quorum"

Everything that changes:

- `controlPlaneEndpoint`
- cert upload
- join flags
- LB/VIP
- etcd quorum

Air-gapped Kubernetes setup

step-by-step OFFLINE setup procedure for a **3 Control Plane + 6 Worker Kubernetes cluster**, where:

- **Production nodes have NO internet**
- **Assuming only one jump/build server has internet**
  Server: `172.18.10.30 (Taken as example)`
- This jump server has **SSH/SCP access** to **all production nodes**
- Kubernetes version, cri-dockerd, Flannel, HAProxy, Keepalived already decided

This approach is commonly called: **"Air-gapped Kubernetes setup using a bastion / build node"**

I'll structure this so you can **directly convert it into documentation or SOP**.

High-Level Architecture (Offline Model)

Internet -----> [172.18.10.30] - Build / Bastion Node | (scp / rsync------> Production Network (NO internet) | CP-1 CP-2 CP-3 | - | WN-1 ... WN-6 |

Rule:

- `172.18.10.30` = **only internet access**
- All packages, images, binaries are **downloaded here first**, then **copied**

OVERALL STRATEGY (IMPORTANT)

We will do everything in **4 stages**:

1. **Prepare offline artifacts on 172.18.10.30**
2. **Copy artifacts to all production servers**
3. **Install OS-level dependencies offline**
4. **Bootstrap Kubernetes cluster**

STAGE 1: Prepare Offline Artifacts (ON 172.18.10.30)

1. Create working directories

```
mkdir -p /opt/k8s-offline/{rpm,images,manifests,binaries}
cd /opt/k8s-offline
```

2. Download OS & Kubernetes RPMs

Required RPMs

- docker-ce
- docker-ce-cli
- containerd.io
- cri-dockerd
- kubeadm
- kubelet
- kubectl
- haproxy
- keepalived
- dependencies

Example:

Command: dnf download --resolve docker-ce docker-ce-cli containerd.io kubeadm-1.33.6 kubelet-1.33.6 kubectl-1.33.6 haproxy keepalived --destdir=./rpm

-> This produces **all required RPMs + dependencies**

3. Download cri-dockerd RPM

Command: wget https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.14/cri-dockerd-0.3.14-1.el9.x86_64.rpm -P ./rpm

4. Pull Kubernetes container images (CRITICAL STEP)

This step avoids **image pull failures later**.

Command: kubeadm config images pull --kubernetes-version v1.33.6 --cri-socket unix:///var/run/cri-dockerd.sock

Then export images:

Command: docker save $(kubeadm config images list --kubernetes-version v1.33.6) -o images/k8s-control-plane-images.tar

5. Download Flannel manifest

Command: curl -fsSL https://github.com/flannel-io/flannel/releases/download/v0.25.6/kube-flannel.yml -o manifests/kube-flannel.yml

At this point you have:

- All RPMs
- All Kubernetes images
- Flannel manifest

- cri-dockerd package

STAGE 2: Copy Artifacts to Production Nodes

On **172.18.10.30**, run:

Command:
for NODE in cp1 cp2 cp3 wn1 wn2 wn3 wn4 wn5 wn6
do
scp -r /opt/k8s-offline root@$NODE:/opt/
done

-----> Every node now has:

/opt/k8s-offline/
├── rpm/
├── images/
└── manifests/

STAGE 3: Offline Installation on ALL Production Nodes

6. Install RPMs offline (ALL nodes)

Commands:
cd /opt/k8s-offline/rpm
dnf install -y *.rpm

---> This installs:

- Docker
- cri-dockerd
- kubeadm
- kubelet
- kubectl (only needed on CPs)
- HAProxy / Keepalived (CPs only)

7. Enable and start services

Commands: systemctl enable --now docker
systemctl enable --now cri-docker
systemctl enable --now kubelet

8. Load Kubernetes images offline (ALL nodes)

Command: docker load -i /opt/k8s-offline/images/k8s-control-plane-images.tar

----> **This is mandatory** - otherwise kubeadm will fail.

9. Kernel & sysctl configuration (ALL nodes)

Commands:
swapoff -a
sed -ri '/swap/d' /etc/fstab
modprobe br_netfilter

cat <<EOF >/etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
EOF

sysctl --system

STAGE 4: HA + Kubernetes Bootstrap

10. Setup HA (CP-1 and CP-2 only)

Keepalived

Configure `/etc/keepalived/keepalived.conf` (as per your doc)

Shell
systemctl enable --now keepalived

HAProxy

Configure `/etc/haproxy/haproxy.cfg`

Shell
systemctl enable --now haproxy

---> Verify VIP exists:
ip a | grep 10.100.152.100

11. kubeadm init (ONLY CP-1)

Use your existing **kubeadm config file**:

kubeadm init --config kubeadm-config.yaml

12. Configure kubectl (CP-1)

mkdir -p ~/.kube
cp /etc/kubernetes/admin.conf ~/.kube/config

13. Upload certificates (CP-1)

kubeadm init phase upload-certs --upload-certs

kubeadm token create --print-join-command
----> Save:

- token
- cert-key
- hash

14. Install Flannel (CP-1)

kubectl apply -f /opt/k8s-offline/manifests/kube-flannel.yml
---> Wait for CNI pods to be Ready:
kubectl -n kube-flannel get pods

15. Join CP-2 and CP-3

kubeadm join 10.100.152.100:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash> --control-plane --certificate-key <cert-key> --cri-socket unix:///var/run/cri-dockerd.sock

16. Join Worker Nodes (ALL WNs)

kubeadm join 10.100.152.100:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash> --cri-socket unix:///var/run/cri-dockerd.sock

FINAL VERIFICATION

kubectl get nodes -o wide
kubectl get pods -A

You should see:

- 3 `control-plane` nodes

- 6 Ready workers
- All system pods Running

KEY OFFLINE-SPECIFIC PITFALLS TO REMEMBER

Issue	Cause
ImagePullBackOff	Images not preloaded
kubeadm init failure	Missing images
CNI not ready	Flannel not copied
Join hangs	kubelet not running

FINAL SUMMARY (One-Line)

172.18.10.30 becomes your Kubernetes "factory".
Everything the cluster needs is built there, shipped, and assembled offline.

Production Monitoring and Envoy Setup (Helm)

Scope

This document provides a production-ready procedure to deploy the **observability stack** and **Envoy Gateway** on Kubernetes using **Helm**, aligned to the already-validated TEST/UAT design. It covers two deployment models:

1. **Production with internet access**
2. **Production without internet access (air-gapped)**

The baseline below keeps the same chart/release pattern already used in UAT so drift stays minimal during production rollout. The UAT design uses Helm **v3.19.0**, namespaces *monitoring* and *envoy-gateway-system*, and the mandatory install order **Prometheus → Loki → Tempo → Alloy → Envoy Gateway** because Alloy sends telemetry to the backends and those backends must exist first.

Baseline versions (same as UAT)

Component	Purpose	Helm Chart	Chart Version	App Version	Release	Namespace
Prometheus Stack	Metrics, alerting, Grafana	prometheus-community/kube-prometheus-stack	80.9.1	v0.87.1	prometheus	monitoring
Loki	Log aggregation	grafana/loki	6.49.0	3.6.3	loki	monitoring
Tempo	Distributed tracing	grafana/tempo	1.24.1	2.9.0	tempo	monitoring
Alloy	Metrics / logs / traces agent	grafana/alloy	1.5.1	v1.12.1	alloy	monitoring
Envoy Gateway	Gateway API ingress / routing	envoyproxy/gateway-helm	v1.6.1	v1.6.1	eg	envoy-gateway-system

These versions and release names match the validated UAT reference that you shared.

Production recommendations before install

1) Keep Helm version pinned

Install Helm on CI/admin nodes only (not inside the cluster) and keep the version pinned to **v3.19.0** to stay aligned with your TEST/UAT validation.

2) Keep chart versions pinned for the first PROD rollout

For the initial production release, use the **same chart versions as UAT** to reduce drift. Upgrade only after a dedicated validation cycle.

3) Separate PROD values from UAT values

Recommended repo structure:

```
monitoring-helm-values/
  prod/
    prometheus-values.yaml
    loki-values.yaml
    tempo-values.yaml
    alloy-values.yaml
envoy-gateway/
  prod/
    gateway-values.yaml
     gateway.yaml
     httproutes/
```

4) Production-specific values to review

At minimum, review these before PROD:

- PVC size / storage class for Prometheus, Loki, Tempo
- Retention for metrics, logs, traces
- Requests / limits
- Anti-affinity / topology spread for HA
- PodDisruptionBudgets
- Grafana admin secret / SSO / TLS
- Alertmanager routing (email / Teams / webhook)
- Alloy endpoints to Prometheus / Loki / Tempo
- Envoy service type (*LoadBalancer* via MetalLB or on-prem LB, or *NodePort* if required)
- Static external IP strategy for Envoy
- NetworkPolicies if enforced
- Image registry overrides for air-gapped deployment

5) CRD handling in production

The *kube-prometheus-stack* chart creates CRDs but Helm v3 does **not** update chart-installed CRDs automatically during upgrades, so CRD changes must be handled deliberately during future upgrades.

Envoy Gateway's Helm chart includes both **Gateway API CRDs** and **Envoy Gateway CRDs**. If you want stricter lifecycle control in PROD, install/manage CRDs separately and then install the chart with *--skip-crds*; otherwise, keep the same behavior as UAT for parity.

Common pre-checks (both online and air-gapped)

Run these from the admin/CI node before installing charts:

```
helm version
kubectl version
kubectl cluster-info
kubectl get nodes -o wide
kubectl get ns
kubectl get sc
kubectl get pods -n kube-system
```

Expected readiness before installing monitoring:

- Kubernetes cluster healthy
- *monitoring* namespace available (or will be created by Helm)
- persistent storage class available for Prometheus/Loki/Tempo
- MetalLB or equivalent available if Envoy will use *LoadBalancer*
- Gateway API CRDs approach decided (chart-managed vs separately managed)
- DNS / certificates / FQDN decisions finalized for Grafana and gateway endpoints

---

Option 1 — Production with internet access

Step 1: Add Helm repositories

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo add envoyproxy https://charts.envoyproxy.io
```

```
helm repo update
```

The Prometheus stack is installed from *prometheus-community*, Loki/Tempo/Alloy from *grafana*, and Envoy Gateway from *envoyproxy*, which matches your UAT pattern.

Step 2: Create namespaces

```
kubectl create namespace monitoring --dry-run=client -o yaml | kubectl apply -f -
kubectl create namespace envoy-gateway-system --dry-run=client -o yaml | kubectl apply -f -
```

Step 3: Install Prometheus first

```
helm upgrade --install prometheus prometheus-community/kube-prometheus-stack \
  --version 80.9.1 \
  -n monitoring \
  -f monitoring-helm-values/prod/prometheus-values.yaml \
  --create-namespace
```

The chart installs the kube-prometheus stack and its dependent components such as kube-state-metrics, node-exporter, and Grafana.

Validate

```
kubectl get pods -n monitoring
helm list -n monitoring
```

Step 4: Install Loki

```
helm upgrade --install loki grafana/loki \
  --version 6.49.0 \
  -n monitoring \
  -f monitoring-helm-values/prod/loki-values.yaml \
  --create-namespace
```

Grafana's Loki chart supports multiple deployment modes; for production, review whether your values keep a simple deployment or move toward a more HA/scalable mode. Grafana documents monolithic and microservices deployment patterns and recommends microservices mode for the best production experience.

Validate

```
kubectl get pods -n monitoring
kubectl logs -n monitoring statefulset/loki
```

Step 5: Install Tempo

```
helm upgrade --install tempo grafana/tempo \
  --version 1.24.1 \
  -n monitoring \
  -f monitoring-helm-values/prod/tempo-values.yaml \
  --create-namespace
```

Validate

```
kubectl get pods -n monitoring
kubectl logs -n monitoring deployment/tempo
```

Step 6: Install Alloy after all backends are ready

```
helm upgrade --install alloy grafana/alloy \
  --version 1.5.1 \
  -n monitoring \
  -f monitoring-helm-values/prod/alloy-values.yaml \
  --create-namespace
```

Alloy must come **after** Prometheus/Loki/Tempo because it forwards telemetry to those backends. That install order is part of your validated UAT design.

Validate

```
kubectl get pods -n monitoring
kubectl logs -n monitoring daemonset/alloy
```

Step 7: Install Envoy Gateway

Option A — Same as UAT (chart manages CRDs)

```
helm install eg envoyproxy/gateway-helm \
  --version v1.6.1 \
  -n envoy-gateway-system \
  --create-namespace
```

Option B — Recommended PROD pattern if you want CRD lifecycle control

1. Pre-install CRDs (Gateway API + Envoy Gateway CRDs) using your approved process.
2. Install Envoy Gateway with:

```
helm install eg envoyproxy/gateway-helm \
  --version v1.6.1 \
  -n envoy-gateway-system \
  --create-namespace \
  --skip-crds
```

Envoy Gateway's chart supports installing normally or with *--skip-crds* when CRDs are managed separately. The chart bundles Gateway API CRDs and Envoy Gateway CRDs.

Validate

```
kubectl get pods -n envoy-gateway-system
kubectl get gatewayclasses
kubectl get gateways -A
kubectl get httproutes -A
```

Step 8: Apply Gateway and HTTPRoute resources

Use the same path pattern already used in UAT, but keep separate production manifests/values.

```
kubectl apply -f envoy-gateway/prod/gateway.yaml
kubectl apply -f envoy-gateway/prod/httproutes/
```

Step 9: Final validation

```
helm ls -A
helm list -n monitoring
```

```
kubectl get pods -n monitoring
kubectl get pods -n envoy-gateway-system
kubectl get gateways -A
kubectl get gatewayclasses
kubectl get httproutes -A
```

Step 10: Rollback commands

```
helm history prometheus -n monitoring
helm rollback prometheus <revision> -n monitoring

helm history loki -n monitoring
helm rollback loki <revision> -n monitoring

helm history tempo -n monitoring
helm rollback tempo <revision> -n monitoring

 helm history alloy -n monitoring
 helm rollback alloy <revision> -n monitoring

 helm history eg -n envoy-gateway-system
 helm rollback eg <revision> -n envoy-gateway-system
```

Option 2 — Production without internet access (air-gapped)

Recommended model

Use one **internet-connected staging/admin host** (for example your Jenkins/admin node) to:

1. download chart archives,
2. collect values files,
3. identify and mirror all container images,
4. transfer charts + values + images into Production.

Helm can download charts locally with *helm pull*, and Helm can install directly from a **local** *.tgz* **chart archive** or unpacked chart directory.

Step 1: Prepare offline bundle on connected host

```
mkdir -p /opt/prod-helm-offline/{charts,values,rendered,images}
```

```
cd /opt/prod-helm-offline

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo add envoyproxy https://charts.envoyproxy.io
helm repo update
```

Step 2: Download chart packages

```
helm pull prometheus-community/kube-prometheus-stack --version 80.9.1 -d charts/
helm pull grafana/loki --version 6.49.0 -d charts/
helm pull grafana/tempo --version 1.24.1 -d charts/
helm pull grafana/alloy --version 1.5.1 -d charts/
helm pull oci://docker.io/envoyproxy/gateway-helm --version v1.6.1 -d charts/
```

helm pull downloads a chart package locally and can also verify/provenance-check a chart when used with the relevant flags.

> Important: your UAT command uses envoyproxy/gateway-helm via repo, while the official Envoy Gateway docs publish the chart as OCI (oci://docker.io/envoyproxy/gateway-helm). For an air-gapped workflow, the OCI path is usually the cleaner way to pull the chart archive on the connected host.

Step 3: Copy your production values files into the offline kit

```
cp -r /path/to/repo/monitoring-helm-values/prod values/
cp -r /path/to/repo/envoy-gateway/prod values/envoy-gateway-prod
```

Step 4: Render manifests and extract container images

For air-gapped installs, you must also mirror all referenced images. A practical method is to template each chart with the exact values file and extract the image references.

```
helm template prometheus charts/kube-prometheus-stack-80.9.1.tgz \
  -n monitoring -f values/prod/prometheus-values.yaml > rendered/prometheus.yaml

helm template loki charts/loki-6.49.0.tgz \
  -n monitoring -f values/prod/loki-values.yaml > rendered/loki.yaml

```

```
helm template tempo charts/tempo-1.24.1.tgz \
  -n monitoring -f values/prod/tempo-values.yaml > rendered/tempo.yaml

helm template alloy charts/alloy-1.5.1.tgz \
  -n monitoring -f values/prod/alloy-values.yaml > rendered/alloy.yaml

helm template eg charts/gateway-helm-v1.6.1.tgz \
  -n envoy-gateway-system > rendered/envoy.yaml
```

Extract image references:

```
grep -rhoE 'image: *[^ ]+' rendered/ | awk '{print $2}' | sort -u > images/all-images.txt
cat images/all-images.txt
```

Step 5: Mirror images into a private registry (recommended)

A private on-prem registry is the cleanest production pattern for air-gapped Helm deployments. Update all values files to point image repositories at that registry.

Example pattern:

```
image:
  registry: registry-prod.example.com
  repository: grafana/loki
  tag: 3.6.3
```

If you do **not** use a private registry, you would need to preload images on every node/container runtime, which is harder to operate long term.

Step 6: Transfer charts + values + image archives to Production

```
scp -r /opt/prod-helm-offline root@<prod-admin-host>:/opt/
```

Step 7: Install from local chart archives in Production

Helm supports installing from a packaged chart path such as *./chart.tgz*.

Prometheus

```
helm upgrade --install prometheus /opt/prod-helm-offline/charts/kube-prometheus-stack-80.9.1.tgz \
  -n monitoring \
  -f /opt/prod-helm-offline/values/prod/prometheus-values.yaml \
  --create-namespace
```

Loki

```
helm upgrade --install loki /opt/prod-helm-offline/charts/loki-6.49.0.tgz \
  -n monitoring \
  -f /opt/prod-helm-offline/values/prod/loki-values.yaml \
  --create-namespace
```

Tempo

```
helm upgrade --install tempo /opt/prod-helm-offline/charts/tempo-1.24.1.tgz \
  -n monitoring \
  -f /opt/prod-helm-offline/values/prod/tempo-values.yaml \
  --create-namespace
```

Alloy

```
helm upgrade --install alloy /opt/prod-helm-offline/charts/alloy-1.5.1.tgz \
  -n monitoring \
  -f /opt/prod-helm-offline/values/prod/alloy-values.yaml \
  --create-namespace
```

Envoy Gateway

```
helm install eg /opt/prod-helm-offline/charts/gateway-helm-v1.6.1.tgz \
  -n envoy-gateway-system \
  --create-namespace
```

If CRDs were pre-installed separately, add *--skip-crds* for Envoy Gateway.

Step 8: Apply production Gateway/HTTPRoute manifests

```
kubectl apply -f /opt/prod-helm-offline/values/envoy-gateway-prod/gateway.yaml
kubectl apply -f /opt/prod-helm-offline/values/envoy-gateway-prod/httproutes/
```

Step 9: Validate

```
helm ls -A
kubectl get pods -n monitoring
kubectl get pods -n envoy-gateway-system
kubectl get gatewayclasses
kubectl get gateways -A
kubectl get httproutes -A
```

Suggested production hardening changes vs UAT

Prometheus / Grafana / Alertmanager

- enable persistence explicitly
- size storage based on retention and scrape volume
- set *retention* / *retentionSize*
- secure Grafana with ingress/gateway + TLS + SSO
- wire Alertmanager receivers before go-live
- add anti-affinity / topology spread and PDBs for Grafana/Alertmanager/Prometheus

Loki

- use persistent/object storage appropriately
- review ingestion and retention limits

- use production sizing (CPU/memory) and replica strategy
- if log volume is high, review Grafana's production guidance for more scalable deployment modes.

Tempo

- lower retention than metrics/logs
- use persistent/object storage
- review compaction/ingester/storage settings for production scale

Alloy

- pin backend endpoints to production service DNS names
- set relabeling / scrape filters to avoid noisy metrics
- ensure log labels do not explode cardinality

Envoy Gateway

- decide whether the Service will be *LoadBalancer* (MetalLB/on-prem LB) or *NodePort*
- reserve static IP(s) for north-south traffic
- keep Gateway and HTTPRoute manifests in a separate production folder
- if using only stable Gateway API resources, consider separately managing **standard channel** Gateway API CRDs as a more conservative production model. Gateway API documents both standard and experimental channels, and many controllers can install the CRDs for you.

Final production checklist

- Helm v3.19.0 installed on admin/CI nodes only
- Same chart versions as UAT pinned for initial PROD rollout
- PROD values files created separately from UAT
- StorageClass/PVs validated for Prometheus, Loki, Tempo
- Grafana credentials / TLS / SSO decided
- Alertmanager routes configured
- Alloy endpoints validated
- Envoy external IP / LB strategy finalized
- Gateway/HTTPRoute production manifests ready
- CRD strategy decided (*chart-managed* vs *separate* + *--skip-crds*)
- Rollback procedure documented and tested

- Air-gapped bundle procedure tested from a non-prod environment if PROD has no internet

---

References

- Your validated UAT/prod baseline, install order, chart versions, namespaces, and Helm version.
- Helm *pull* and local install support.
- kube-prometheus-stack CRD behavior in Helm v3.
- Envoy Gateway Helm install and CRD behavior.
- Gateway API standard vs experimental channels.
- Loki production deployment guidance.